

**Documentación Técnica Detallada:
Sistema de Simulación y Optimización de Ruido en
Edificios**

Integrantes:

Juan Manuel Serrano Rodríguez
David Santiago Torres Mendieta

Universidad Distrital Francisco José de Caldas
Facultad de Ingeniería
Ciencias de la Computación II
Bogotá, 2025

Índice

1. Introducción

1.1. Descripción General del Sistema

Este sistema permite simular y optimizar la distribución de espacios en un edificio considerando la propagación del ruido. El programa:

- Lee una configuración inicial desde un archivo JSON
- Simula la propagación física del ruido entre espacios
- Optimiza la distribución para minimizar problemas acústicos
- Genera visualizaciones 3D interactivas
- Produce reportes detallados con recomendaciones

2. Estructura del Sistema

2.1. Carga Inicial del Edificio

El sistema inicia cargando la configuración del edificio desde un archivo JSON que contiene:

- **Datos de los espacios:**
 - Nombre y tipo de espacio
 - Posición (x,y,z) en el edificio
 - Nivel de piso
 - Características físicas (paredes, ventanas, puertas)
 - Niveles de ruido base
 - Frecuencias características

- **Proceso de carga:**

```
1  def inicializar_nodos(self):
2      try:
3          with open('edificio.json', 'r', encoding='utf
-8') as f:
4              data = json.load(f)
5      except FileNotFoundError:
6          QMessageBox.critical(self, "Error",
7                               "El archivo 'edificio.json' no se encontr
\\'{o}.'.")
8          sys.exit(1)
9
```

2.2. Modelo de Nodos

Cada espacio se representa como un nodo con las siguientes características:

- **Atributos Físicos:**
 - **name:** Identificador único del espacio
 - **tipo:** Categoría (aula, biblioteca, etc.)
 - **position:** Coordenadas (x,y,z)
 - **piso:** Nivel en el edificio
 - **pared:** Presencia de paredes (booleano)
 - **ventana:** Presencia de ventanas (booleano)
 - **puerta:** Presencia de puertas (booleano)
- **Atributos Acústicos:**
 - **ruido:** Nivel base en decibelios (dB)
 - **frecuencia:** Frecuencia característica (Hz)
 - **es_fuente:** Si es generador activo de ruido
- **Conexiones:**
 - Lista de nodos vecinos
 - Conexiones bidireccionales automáticas
 - Sensores de medición asociados

2.3. Modelado de Propagación del Ruido

El cálculo del ruido total en cada espacio sigue un modelo físico que considera múltiples factores:

$$R_{total} = R_{base} + \sum_{i=1}^n (R_i \times A_i \times F_i \times E_i) \quad (1)$$

Donde cada término representa:

- R_{base} : Ruido propio del espacio (si es fuente)
- R_i : Ruido del vecino i

- A_i : Atenuación por distancia según ley cuadrática inversa
- F_i : Factor de atenuación por frecuencia
- E_i : Efectos estructurales combinados

2.3.1. Atenuación por Distancia

La atenuación sigue la ley cuadrática inversa:

$$A_d = \frac{1}{d^2} \quad (2)$$

Donde d es la distancia euclidiana entre espacios:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \quad (3)$$

2.3.2. Factor de Frecuencia

La atenuación por frecuencia se calcula como:

$$F_f = \min(1, 0, \frac{1000}{f}) \quad (4)$$

Donde f es la frecuencia en Hz. Este factor modela la mayor atenuación de altas frecuencias.

3. Modelo de Clases

3.1. Diagrama de Clases

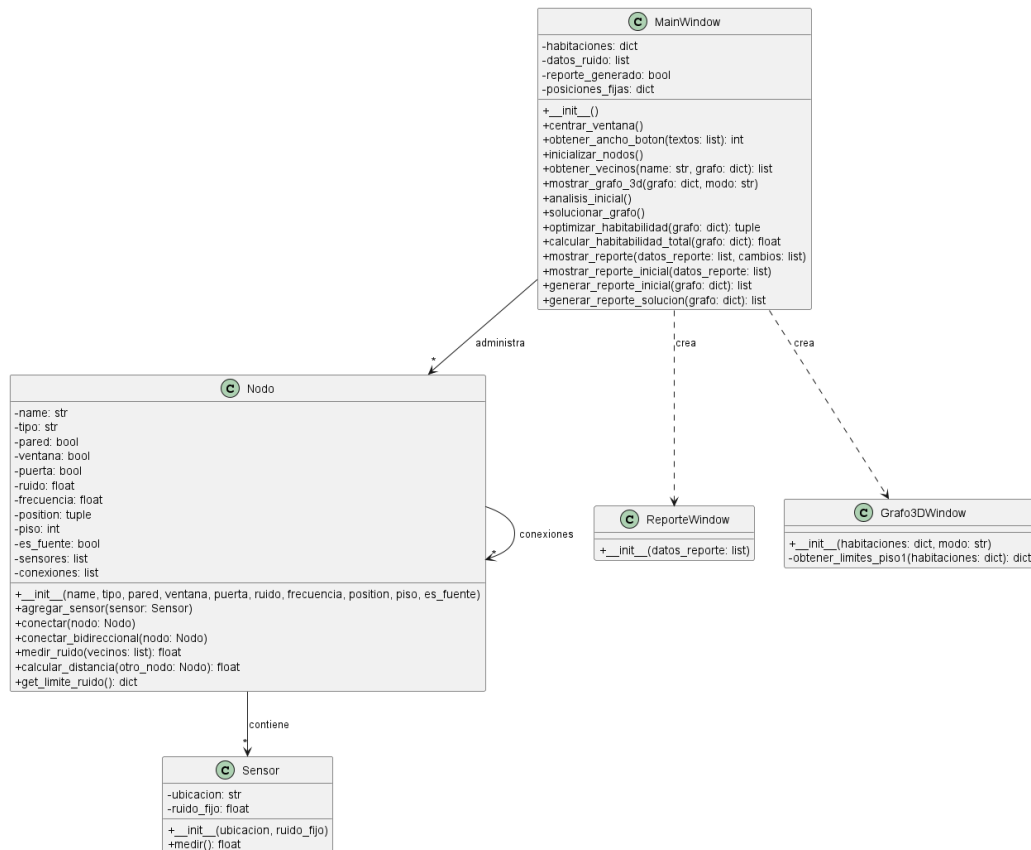


Figura 1: Diagrama de Clases del Sistema

3.2. Descripción de Clases

■ Nodo

- Representa un espacio en el edificio
- Gestiona propiedades físicas y acústicas
- Implementa métodos de cálculo de ruido
- Mantiene conexiones con otros nodos

■ Sensor

- Simula mediciones de ruido
- Puede tener valores fijos o calculados
- **MainWindow**
 - Interfaz principal del sistema
 - Gestiona la visualización y optimización
 - Coordina la generación de reportes
- **ReporteWindow**
 - Visualiza resultados de análisis
 - Muestra recomendaciones específicas
- **Grafo3DWindow**
 - Genera visualizaciones 3D del edificio
 - Representa niveles de ruido mediante colores
 - Muestra conexiones entre espacios

4. Interacción entre Componentes

4.1. Flujo de Datos

- **Inicialización:**
 1. Carga de configuración desde JSON
 2. Creación de nodos y sensores
 3. Establecimiento de conexiones
- **Análisis Inicial:**
 1. Medición de niveles de ruido
 2. Generación de reporte inicial
 3. Visualización 3D del estado
- **Proceso de Optimización:**
 1. Copia del estado inicial
 2. Aplicación de recocido simulado
 3. Generación de reporte de solución
 4. Visualización del resultado optimizado

4.2. Gestión de Eventos

- **Eventos de Usuario:**
 - Inicio de análisis
 - Solicitud de optimización
 - Interacción con visualizaciones 3D
- **Eventos del Sistema:**
 - Actualización de estados
 - Generación de reportes
 - Recálculo de propagación de ruido

5. Consideraciones de Implementación

5.1. Manejo de Recursos

- **Gestión de Memoria:**
 - Uso de copias profundas para preservar estados
 - Limpieza de recursos gráficos
- **Eficiencia:**
 - Optimización de cálculos de propagación
 - Caché de resultados intermedios
 - Actualización selectiva de visualizaciones

5.2. Extensibilidad

- **Puntos de Extensión:**
 - Nuevos tipos de espacios
 - Algoritmos alternativos de optimización
 - Formatos adicionales de reporte
- **Modificación de Parámetros:**
 - Límites de ruido configurables
 - Factores de atenuación ajustables
 - Criterios de optimización personalizables

6. Algoritmo de Optimización

6.1. Recocido Simulado

El algoritmo utiliza recocido simulado para optimizar la distribución de espacios. Este proceso imita el enfriamiento gradual de materiales para encontrar un estado óptimo.

6.2. Parámetros Principales

- **Temperatura Inicial (T_0):** 100.0
 - Controla la probabilidad de aceptar soluciones peores
 - Valor alto inicial permite exploración amplia
- **Factor de Enfriamiento (α):** 0.95
 - Reduce gradualmente la temperatura
 - $T_{nueva} = T_{actual} \times 0,95$
- **Iteraciones:** 1000
 - Número de intentos de optimización
 - Balance entre tiempo y calidad de solución

6.3. Proceso de Optimización

6.3.1. 1. Selección de Nodos

- **Exclusiones Específicas:**
 - Pasillos: No se consideran para intercambio
 - Recepción: Mantiene su ubicación fija
 - Razón: Son espacios estructurales críticos
- **Nodos Problemáticos:**
 - Se identifican espacios que exceden límites de ruido
 - 70 % de probabilidad de seleccionar nodo problemático
 - 30 % de selección aleatoria para diversidad

```

1     nodos_problematicos = [
2         nodo for nodo in nuevo_grafo.values()
3         if nodo.tipo != "pasillo" and
4           nodo.tipo != "recepcion" and
5           nodo.medir_ruido(vecinos) >
6           nodo.get_limite_ruido()['limite_cercano']
7     ]
8

```

6.3.2. 2. Matriz de Compatibilidad

Define qué tipos de espacios pueden intercambiarse:

Espacio	Compatible con	Justificación
Biblioteca	Oficina, Reuniones	Espacios que requieren silencio relativo
Aula	Laboratorio, Auditorio	Espacios educativos con niveles similares
Cafetería	Reuniones	Espacios sociales con ruido moderado
Laboratorio	Aula, Oficina	Espacios de trabajo controlado
Oficina	Biblioteca, Reuniones	Ambientes de trabajo tranquilos
Auditorio	Aula	Espacios grandes con acústica similar

6.3.3. 3. Proceso de Intercambio

Cuando se seleccionan dos nodos compatibles:

1. Intercambio de Propiedades:

- Tipo de espacio
- Nivel de ruido base
- Condición de fuente
- Posición física

2. Evaluación del Cambio:

- Se calcula nuevo puntaje de habitabilidad
- Se compara con estado anterior

```

1     # Intercambio de propiedades
2     nodo1.tipo, nodo2.tipo = nodo2.tipo, nodo1.tipo
3     nodo1.ruido, nodo2.ruido = nodo2.ruido, nodo1.ruido
4     nodo1.es_fuente, nodo2.es_fuente = nodo2.es_fuente,
      nodo1.es_fuente

```

```

5     nodo1.position, nodo2.position = nodo2.position,
    nodo1.position
6

```

7. Visualización y Sistema de Reportes

7.1. Visualización 3D

La visualización del edificio se realiza mediante la clase `Grafo3DWindow`, que implementa:

7.1.1. Construcción del Grafo

■ Creación de Nodos:

```

1     G = nx.Graph()
2     for name, nodo in habitaciones.items():
3         G.add_node(name, tipo=nodo.tipo,
4                     ruido=nodo.medir_ruido(vecinos))
5

```

■ Establecimiento de Conexiones:

1. **Por Piso:** Conecta todos los nodos del mismo nivel
2. **Entre Pisos:** Conecta nodos si la distancia ≤ 7 unidades
3. **Visualización:** Líneas punteadas con grosor inversamente proporcional a la distancia

7.1.2. Código de Colores

Color	Estado	Significado
Verde	Adecuado	Nivel de ruido dentro de límites
Amarillo	Cerca	Próximo a exceder límites
Rojo	Excede	Supera límites permitidos

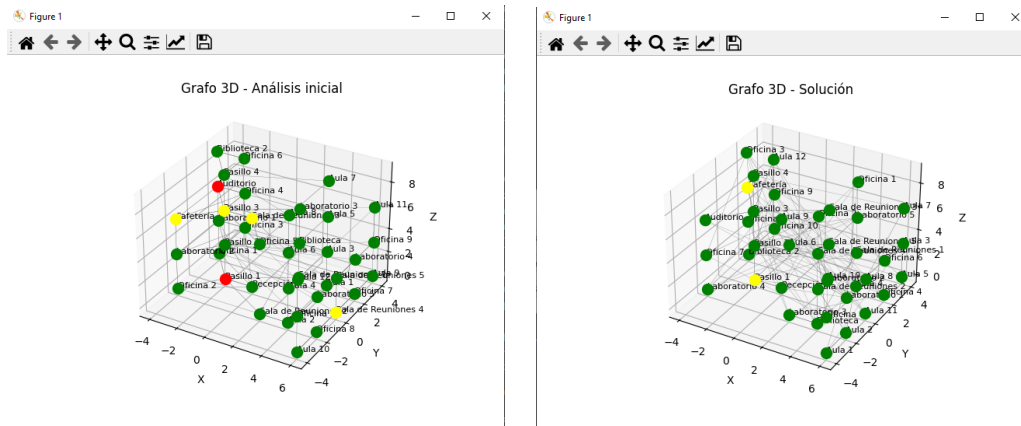


Figura 2: Comparativa: Gráfica Inicial (izquierda) y Gráfica Optimizada (derecha)

8. Sistema de Reportes

8.1. Generación de Reportes

El sistema implementa dos tipos de reportes:

8.1.1. 1. Reporte Inicial

- Genera un listado del estado de cada espacio
- Muestra sugerencias rápidas de mitigación

```

1 def generar_reporte_inicial(self, grafo):
2     datos_reporte = []
3     for name, nodo in grafo.items():
4         nivel = nodo.medir_ruido(vecinos)
5         estado = determinar_estado(nivel, limites)
6         recomendacion = obtener_recomendacion(
7             tipo_espacio, estado)
8         datos_reporte.append(
9             (name, nivel, estado, recomendacion))

```



Figura 3: Pantallazo del Reporte Inicial

Recomendaciones Específicas por Tipo:

■ Biblioteca:

- Excede: Implementar zonas de silencio...”
- Cerca: Reforzar políticas de silencio...”
- Adecuado: Continuar con políticas actuales...”

[Se incluyen todos los tipos de espacios con sus recomendaciones específicas]

8.1.2. 2. Reporte de Solución

■ Contenido:

- Lista de cambios realizados
- Estado final de cada espacio
- Recomendaciones de mejora

- [X] Rojo: Excede límites
- [!] Amarillo: Cerca del límite
- [V] Verde: Nivel adecuado

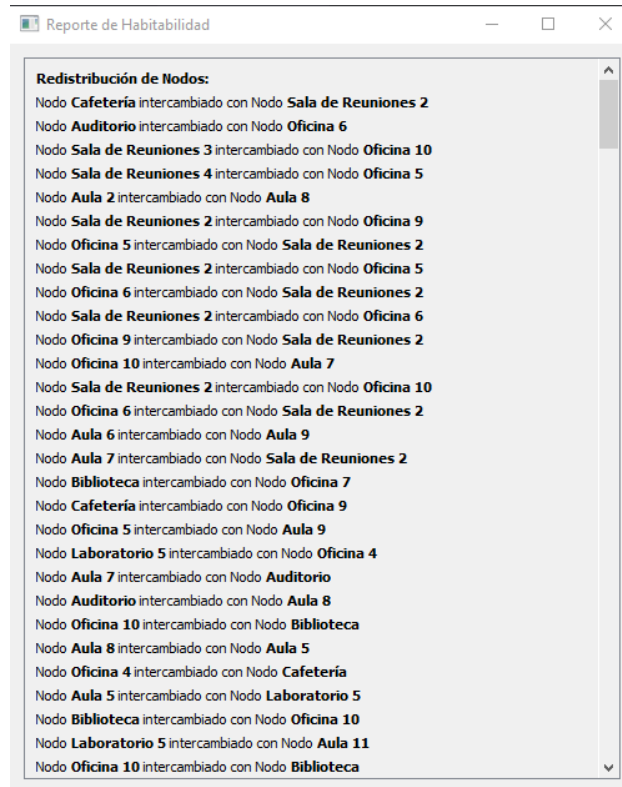


Figura 4: Pantallazo del Reporte Final con la Solución



Figura 5: Pantallazo del Reporte Final con la Solución

9. Interfaz de Usuario

9.1. Ventana Principal

- Botones Principales:
 1. Análisis Inicial
 2. Solucionar Grafo
 3. Salir
- Vista de la Interfaz Principal:

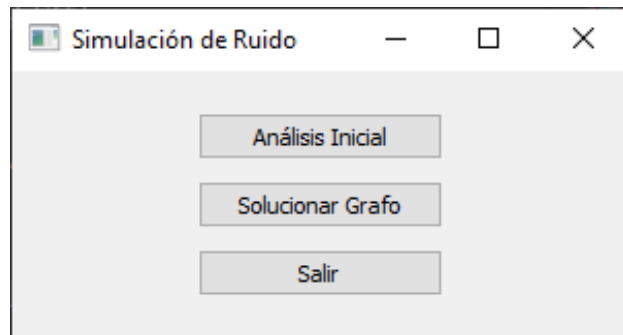


Figura 6: Interfaz Principal del Sistema